RAG для психологии отношений

Отчёт по домашней работе №1 · Чиркова Анна, MFAI-01

1. Данные

Данные я взяла из PDF-статей. Перед работой я убрала разметку страниц и служебную информацию, чтобы в поиск попадал только основной текст. Тема психологии мне в целом интересна, поэтому корпус я решила собрать именно про близкие отношения.

Цель работы — понять, какие настройки сильнее всего влияют на поиск по этому корпусу. По сути, я последовательно меняла отдельные части RAG: границы chunks, способ поиска и количество текста, которое получает генеративная модель. В корпусе 10 документов; средний размер — 3229,8 слова, но документы заметно отличаются по длине.

Для embeddings я выбрала intfloat/multilingual-e5-small, потому что она поддерживает русский язык. Текст chunks подавался с префиксом passage:, а вопрос — с query:. Embeddings нормализованы, а в Chroma использована cosine distance. Чтобы оценить поиск, я заранее составила 10 вопросов и сама отметила chunks с правильными ответами. Recall@k показывает, какая доля этих chunks попала в первые k результатов.

2. Гипотеза 1: overlap на границах

Сначала я сравнила chunks по 200 токенов без пересечения и с overlap 40 токенов (20%). Я ожидала, что повторение части текста поможет в случаях, когда нужная мысль оказалась ровно на границе двух chunks.

Вариант	Chunks	Recall@1	Recall@3	Recall@5	Память
Без overlap	300	0.25	0.60	0.70	0.44 МБ
Overlap 40	369	0.20	0.55	0.75	0.54 МБ

Результат получился не таким однозначным, как я ожидала. Recall@5 вырос с 0.70 до 0.75, но количество chunks увеличилось с 300 до 369, то есть на 23%. Уже здесь я засомневалась, оправдывает ли небольшой прирост дополнительные затраты памяти и вычислений. Overlap помог на q2 и q8, на большинстве вопросов ничего не изменил, а на q4 даже ухудшил результат. Получается, что overlap полезен в отдельных случаях, но сам по себе не гарантирует лучший поиск.

3. Гипотеза 2: смысловое деление

Во второй гипотезе я перестала делить текст просто каждые 200 токенов и попробовала искать смысловые границы. Документ делился на предложения, а для соседних предложений считалась cosine similarity. Порогом стал предложенный в задании 25-й перцентиль: если сходство было ниже него, начинался новый chunk. Получилось 420 chunks с медианой 90 токенов. Фрагменты длиннее 450 токенов я дополнительно делила, чтобы E5 не обрезала их.

Способ chunking	Recall@1	Recall@3	Recall@5
Fixed без overlap	0.250	0.600	0.700
Fixed с overlap	0.200	0.550	0.750
Semantic	0.467	0.633	0.658

На маленьких значениях k смысловое деление действительно помогло. Особенно заметна разница при k=1: Recall вырос до 0.467, тогда как у fixed-size вариантов он составлял 0.250 и 0.200. Но при k=5 semantic chunking дал 0.658 и уступил fixed-size. То есть первый найденный фрагмент чаще был подходящим по смыслу, но общая полнота поиска лучше не стала. Кроме того, даже при смысловом делении всё равно пришлось контролировать длину chunks.

4. Гипотеза 3: Dense, BM25 и RRF

Здесь я сравнивала уже не способы деления текста, а способы поиска по готовым chunks. Dense ищет по смысловой близости embeddings, BM25 — по точным словам, числам и сокращениям, а RRF объединяет два рейтинга по формуле 1 / (60 + rank). Для сравнения я взяла 5 обычных смысловых вопросов и 5 точных запросов с редкими обозначениями.

Тип вопросов	Метод	Recall@1	Recall@3	Recall@5
Обычные	BM25	0.50	0.50	0.50
Обычные	Dense	0.00	0.50	0.90
Обычные	RRF	0.10	0.70	0.70
Точные	BM25	0.90	0.90	0.90
Точные	Dense	0.50	0.80	0.80
Точные	RRF	0.50	0.70	0.90

Гипотеза подтвердилась частично. BM25 лучше ловит точные записи вроде r=0,532, ACRS и Pphubbing, а Dense — вопросы, где важен общий смысл. На обычных вопросах Recall@5 у Dense равен 0.90; на точных такой же результат показали BM25 и RRF. В среднем победил Dense — 0.85 против 0.80 у RRF и 0.70 у BM25. Получается, что гибрид не становится лучшим автоматически: на обычных вопросах BM25 иногда добавлял шум.

5. Гипотеза 4: влияние top-k

В последней гипотезе я проверяла, сколько найденных chunks передавать модели. Для retrieval использовался Dense, а ответы генерировала локальная Qwen2.5-0.5B-Instruct. Я выбрала небольшую открытую модель, чтобы обойтись без платного API. Для 10 вопросов были получены ответы при k = 1, 3, 5 и 10. Оценка учитывала ключевые факты и опору на переданный контекст.

top-k	Средняя оценка	Полнота фактов	Опора на контекст	Рисковые ответы
1	2.4	0.267	0.590	50%
3	2.8	0.317	0.727	20%
5	3.4	0.483	0.785	0%
10	3.0	0.383	0.809	10%

Лучшим оказался top-k=5. При k=1 модели часто не хватало части ответа, а при k=10 появлялся лишний текст. Самым показательным для меня оказался q8: нужный chunk был найден, но модель всё равно не назвала 24% и 7,7%. Значит, хороший retrieval ещё не гарантирует хороший ответ: модель должна правильно вытащить факт из контекста.

6. Итог и ограничения

Для моих 10 документов и вопросов лучшей настройкой стали fixed chunks по 200 токенов с overlap 40, Dense retrieval и top-k=5. Но на точных запросах Recall@5 у BM25 равен 0.90. Если в системе будет много кодов, чисел или редких названий, я бы оставила BM25 и дополнительно настраивала гибридный поиск.

Ограничения эксперимента:

- корпус небольшой, поэтому один вопрос заметно меняет среднее значение;
- релевантность chunks размечалась вручную и может зависеть от выбранной границы;
- автоматическая оценка ответа проверяет ключевые факты, но не заменяет полноценную экспертную оценку;
- Qwen2.5-0.5B — компактная модель, поэтому часть ошибок связана с генератором, а не только с retrieval.

Главный вывод для меня: в RAG нельзя улучшать одну часть отдельно. Overlap меняет размер базы, chunking — форму контекста, retrieval — качество поиска, а top-k — объём текста для LLM.